

Tutorial Sheet 5b

Inter-Process Communication

AUTHOR

Operating Systems Teaching and Research Unit

PUBLISHED

28.06.2024

Introduction Questions

Question 1

Pipes and Shared Memory are two concepts for exchanging information between processes. Compare the advantages and disadvantages of these two concepts.

Question 2

In the lecture, it was mentioned that named pipes are very similar to files. What are the main differences between files and named pipes?

Question 3

Scenario: Web Server Configuration

Web servers typically read a configuration file and apply rules based on it while running. To avoid restarting the server every time the configuration changes, we aim to implement a feature that allows us to update the configuration and trigger a reload without restarting.

Discuss the advantages and disadvantages of using different methods of inter-process communication to achieve this:

- Signals
- Shared Memory
- Pipes
- Sockets

Question 4

Scenario: Autonomous Vehicle Control System

In an autonomous vehicle control system, various subsystems must exchange real-time data such as sensor readings, decision outputs, and control commands. The system requires low latency for rapid response to dynamic road conditions, high bandwidth to handle continuous sensor inputs, and effective synchronization to ensure timely decision-making and execution of control commands.

Discuss the advantages and disadvantages of using each method of IPC to achieve efficient data exchange and synchronization between subsystems in an autonomous vehicle control system.

- Signals
- Shared Memory

- Pipes
- Sockets

Tasks

Task 1: Named Pipes

In the lecture, you learned about named pipes as a mechanism for IPC. Write two programs, `sender.c` and `receiver.c`, that communicate with each other through a named pipe. The `sender.c` program should continuously accept user input and then send it to `receiver.c`, which should continuously wait for transmissions and then print them out. Both programs should terminate when `exit` is transmitted.

Task 2: Shared and Synchronized Blocked Stack

Write in C a shared stack supporting the pop and push operations.

Pop is used to remove the element on the top of the stack. If no element is available, wait until one becomes available.

Push is used to add an element on the top of the stack. If the stack is full, `push` should wait until an element can be inserted.

Multiple processes can push and pop at the same time. Your implementation should be safe meaning that multiple processes/threads can execute the code at the same time without race conditions.

```
1  struct Stack {
2      int top; // the index of the element above the top of the stack
3      int size;
4      int *stack;
5      // complete here with synchronization mechanisms
6  };
7
8  struct Stack *init_stack(int size) {
9      // allocate and initialize the stack
10 }
11
12 int pop(struct Stack *stack) {
13     // pop the top element of the stack and return it
14 }
15
16 void push(struct Stack *stack, int value) {
17     // push a new element on the top of the stack
18 }
```